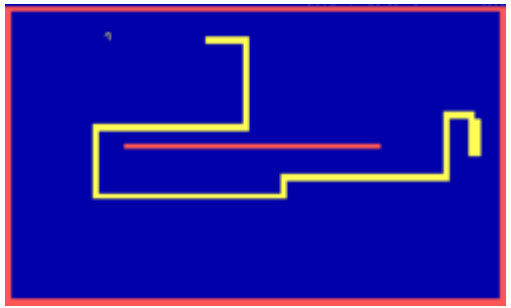
Data Representation

1

CS/COE 0449
Introduction to
Systems Software

Luis Oliveira
(with content borrowed from wilkie and Vinicius Petrucci)



Binary Encoding

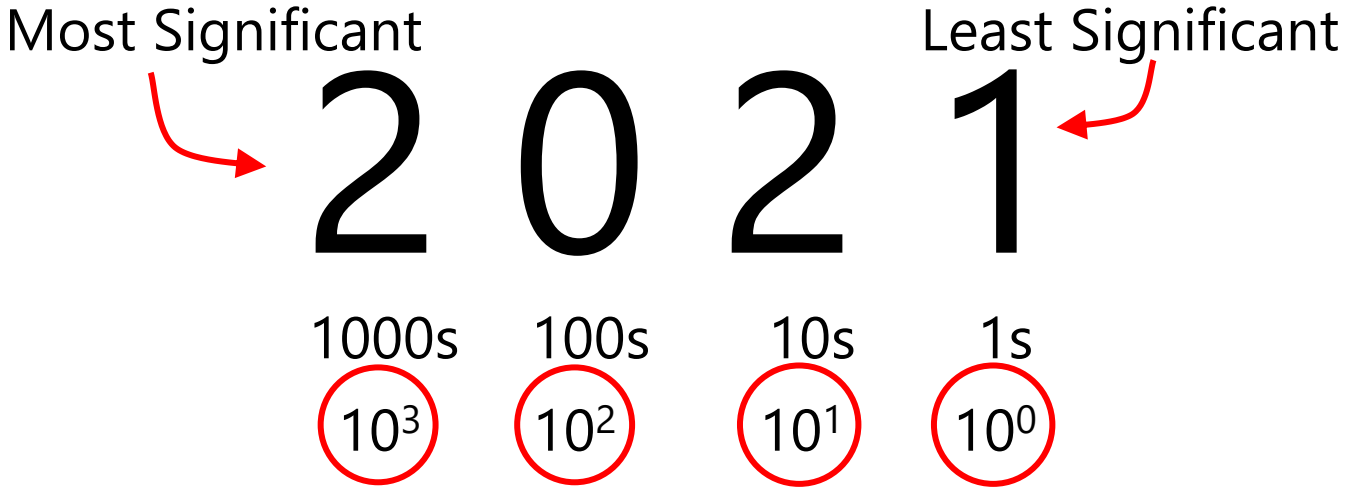
Bits, Bytes, and Nybbles

- The numbers we use are written **positionally**: the position of a digit within the number has a meaning.

$$2021 = \begin{array}{rl} 2000 & = 2 \times 10^3 \\ 000 & = 0 \times 10^2 \\ 20 & = 2 \times 10^1 \\ + \quad 1 & = 1 \times 10^0 \end{array}$$

Positional number systems

- The numbers we use are written **positionally**: the position of a digit within the number has a meaning.



- How many (digits) **symbols** do we have in our number system?
  - 10: 0, 1, 2, 3, 4, 5 ,6 ,7, 8, 9

Range of numbers

Suppose we have a 4-digit numeric display.



- What is the smallest number it can show?
- What is the biggest number it can show?
- How many *different* numbers can it show?
  - 9999 - 0 + 1 = 10,000
  - What power of 10 is 10,000?
    - 10^4

So… with the numbers we use every day…

- A number represented by the digits

$$d_n d_{n-1} \dots d_1 d_0$$

- Has the value

$$d_n \times 10^n + d_{n-1} \times 10^{n-1} + \cdots + d_1 \times 10^1 + d_0 \times 10^0$$

- Using n digits we can represent 10^n different numbers
- The smallest non-negative number representable with n digits is 0
- The largest number representable with n digits is $10^n - 1$
- Using 10 symbols: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9

Numeric Bases

- These 10s keep popping up… and for good reason
- We use a **base-10 (decimal)** numbering system
  - 10 different **symbols**, and each place is a power of 10
- But we can use (almost) any number as a base!
- The most common bases when dealing with computers are **base-2 (binary)**, **base-16 (hexadecimal)**, and (rarely) **base-8 (octal)**
- When dealing with multiple bases, you can write the base as a subscript to be explicit about it:

$$5_{10} = 101_2$$

Making a number system

Using base **B**

- A number represented by the digits

$$d_n d_{n-1} \dots d_1 d_0$$

- Has the value

$$d_n \times \mathbf{B}^n + d_{n-1} \times \mathbf{B}^{n-1} + \cdots + d_1 \times \mathbf{B}^1 + d_0 \times \mathbf{B}^0$$

- Using n digits we can represent $\mathbf{B}^n$ different numbers
- The smallest non-negative number representable with n digits is 0
- The largest number representable with n digits is $\mathbf{B}^n - 1$
- Using **B** symbols



If you use base 0 you don't need to remember any symbols

Binary – Base 2

Let's make a base-2 number system

Using base **2**

- A number represented by the digits

$$d_n d_{n-1} \dots d_1 d_0$$

- Has the value

$$d_n \times \mathbf{2}^n + d_{n-1} \times \mathbf{2}^{n-1} + \cdots + d_1 \times \mathbf{2}^1 + d_0 \times \mathbf{2}^0$$

- Using n digits we can represent $\mathbf{2}^n$ different numbers
- The smallest non-negative number representable with n digits is 0
- The largest number representable with n digits is $\mathbf{2}^n - 1$
- Using **2** symbols: 0, 1

Binary (base-2)

- We call a **B**inary dig**IT** a **bit** – a single 1 or 0
- When we say an *n*-bit number, we mean one with *n* binary digits

MSB LSB

1001 0110 =

2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
128s	64s	32s	16s	8s	4s	2s	1s

$1 \times 128 + 0 \times 64 + 0 \times 32 + 1 \times 16 + 0 \times 8 + 1 \times 4 + 1 \times 2 + 0 \times 1$

$= 150_{10}$

To convert binary to decimal: ignore 0s, add up place values wherever you see a 1.

It's the only odd number!

- Ok! then. Let's go back to decimal for a **bit**

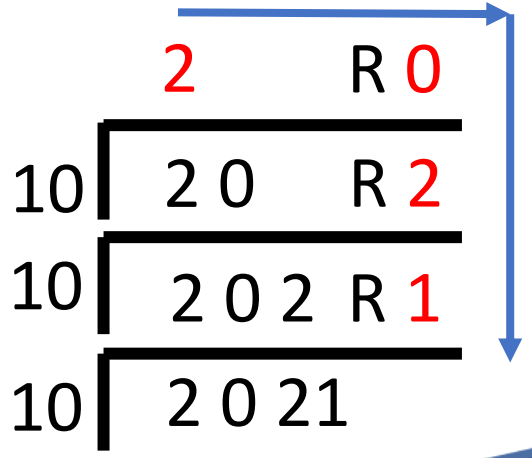
2 0 2 1

How would you extract this number???

How I like to think of it:

When you divide by the BASE you are moving the decimal point in that BASE

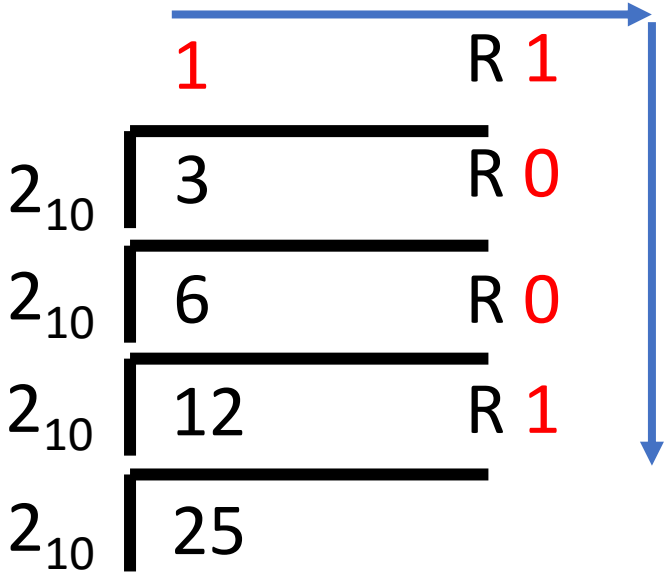
Just divide by 10!



What happens when you divide by 10?

- Turns out that dividing by 10 in any base has the same outcome

$0b10 \rightarrow 2_{10}$



11001_2

Bits, bytes, nibbles, and words

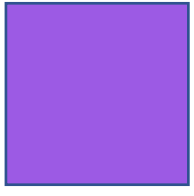
- A ***bit*** is one binary digit, and its unit is **lowercase** b.
- A ***byte*** is an 8-bit value, and its unit is **UPPERCASE** B.
  - This is (partially) why your 30 mega*bit* (Mbps) internet connection can only give you at most 3.57 mega*bytes* (MB) per second!
- A ***nibble*** (also nybble) is 4 bits – half of a byte
  - Corresponds nicely to a single hex digit.
- A ***word*** is the "most comfortable size" of number for a CPU.
- When we say "32-bit CPU," we mean its ***word*** size is 32 bits.
  - This means it can, for example, add two 32-bit numbers at once.
- **BUT WATCH OUT:**
  - Some things (Windows, x86) use **word** to mean **16 bits** and **double word** (or **dword**) to mean **32 bits.**

Why binary? *Whynary?*

- Why indeed?



- What color is this?



Why binary? *Whynary?*

- Why indeed?

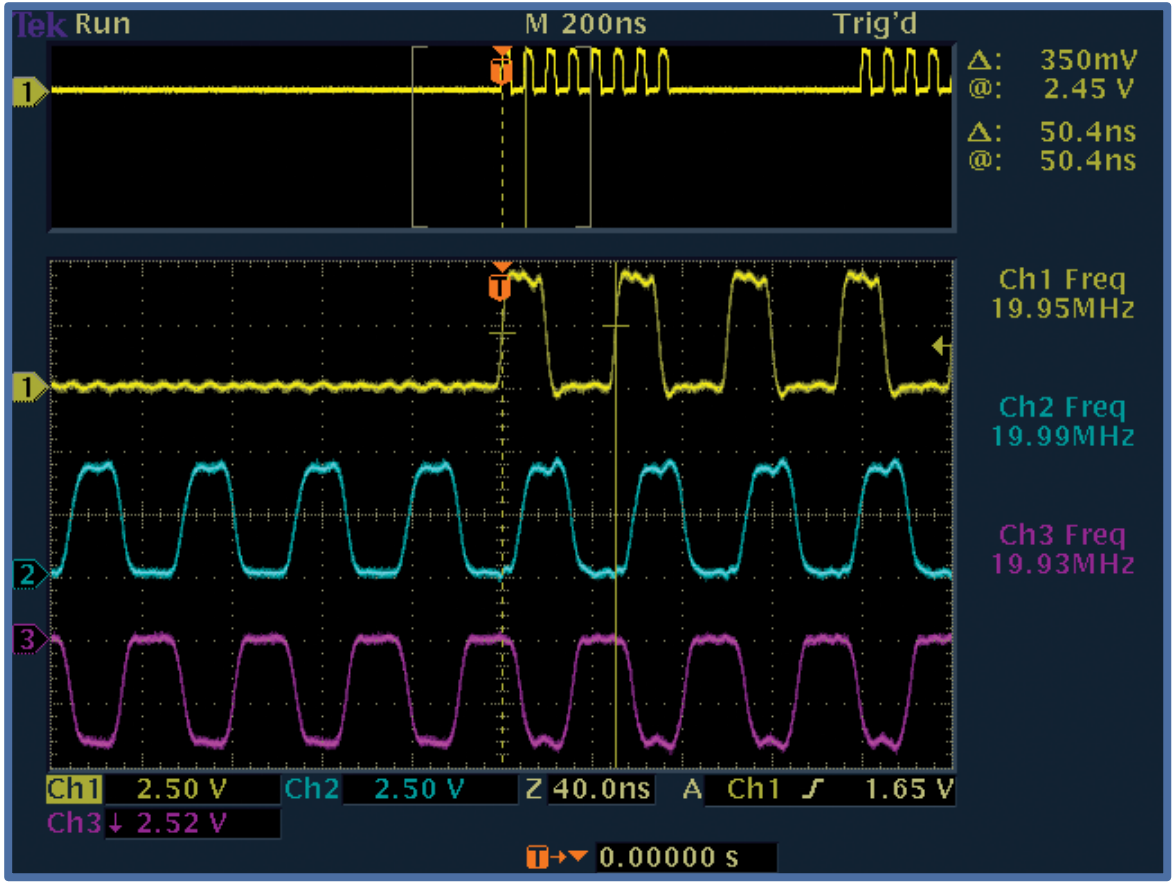


- What color is this?



Binary Representation

- Computers translate electrical signals to either 0 or 1.
- It is relatively easy to devise electronics that operate this way.
- In reality, there is no such thing as “binary” so we often have to approximate and mitigate error.



Oscilloscope visualization of several digital wires. From @computerfact on Twitter.

Everything in a computer is a number

- So, everything on a computer is represented in binary.
  - **everything.**

01100101 01110110 01100101 01110010 01111001 01110100 01101000 01101001 01101110 01100111 00001010 00000000

- Java strings are encoded using **UTF-16**
  - Most letters and numbers in the English alphabet are < 128.
  - "Strings are numbers"
    - **83 116 114 105 110 103 115 32 97 114 101 32 110 117 109 98 101 114 115 0**

- **ASCII** is also pretty common (the best kind of common)
  - That's what we will be using → 8 bit numbers represent characters
  - Letters and numbers (and most/all ascii characters) have the same value as UTF-16

Do try this at home: what does this mean?

- 71 111 111 100 32 74 111 98 0

Hexadecimal – Base 16

The binary shorthand

Hexadecimal

- Binary numbers can get really long, quickly.
  - $3{,}927{,}664_{10} = 11\ 1011\ 1110\ 1110\ 0111\ 0000_2$
- But nice "round" numbers in binary look arbitrary in decimal.
  - $1000000000000000_2 = 32{,}768_{10}$
- This is because 10 is not a power of 2!
- We could use base-4, base-8, base-16, base-32, etc.
  - Base-4 is not much denser than binary
    - e.g. $3{,}927{,}664_{10} = 120\ 3331\ 2323\ 0000_4$
  - Base-32 would require 32 digit symbols. Yeesh.
    - They do, oddly, have their place… but not really in this context.
  - **Base-8** and **base-16** look promising!

Hex	Decimal	Binary
0	0	0000
1	1	0001
2	2	0010
3	3	0011
4	4	0100
5	5	0101
6	6	0110
7	7	0111
8	8	1000
9	9	1001
A	10	1010
B	11	1011
C	12	1100
D	13	1101
E	14	1110
F	15	1111

Hexadecimal or “hex” (base-16)

- Digit symbols after 9 are A-F, meaning 10-15 respectively.
- Usually we call one hexadecimal digit a *hex digit*. No fancy name :(

$$003B\ EE70 =$$

$16^7\ \ 16^6\ \ 16^5\ \ 16^4\ \ \ \ 16^3\ \ 16^2\ \ 16^1\ \ 16^0$

$$\begin{aligned} 0 \times 16^7 + \\ 0 \times 16^6 + \\ 3 \times 16^5 + \\ 11 \times 16^4 + \\ 14 \times 16^3 + \\ 14 \times 16^2 + \\ 7 \times 16^1 + \\ 0 \times 16^0 = \end{aligned}$$

$$3{,}927{,}664_{10}$$

To convert hex to decimal: use a dang calculator lol

Binary to Hex

(animated)

0100	1100	1010	0010	0000	0010	0110	0001
4	C	A	2	0	2	6	1

0x4CA20261

32-bits! (Not so bad…)

Q: Create a random binary string and practice!

Let's make a base-16 number system

Using base **16**

- A number represented by the digits

$$d_n d_{n-1} \dots d_1 d_0$$

- Has the value

$$d_n \times \mathbf{16}^n + d_{n-1} \times \mathbf{16}^{n-1} + \cdots + d_1 \times \mathbf{16}^1 + d_0 \times \mathbf{16}^0$$

- Using n digits we can represent $\mathbf{16}^n$ different numbers
- The smallest non-negative number representable with n digits is 0
- The largest number representable with n digits is $\mathbf{16}^n - 1$
- Using **16** symbols: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F

1111 | 1111

$$1 \times 2^7 + 1 \times 2^6 + 1 \times 2^5 + 1 \times 2^4 + 1 \times 2^3 + 1 \times 2^2 + 1 \times 2^1 + 1 \times 2^0$$

$$1 \times 2^3 + 1 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 = 8 + 4 + 2 + 1 = 15$$

$$(1 \times 2^3 + 1 \times 2^2 + 1 \times 2^1 + 1 \times 2^0) \times 2^4 + 15$$

Factoring

$2^4 = 16$

$$15 \times 16^1 + 15 \times 16^0$$

F | F

This works with any base that is a power of 2

E.g. Base 4=2² Split into groups of 2 bits

Integer Encoding

Casting is Not Just a Witch or Wizard Thing

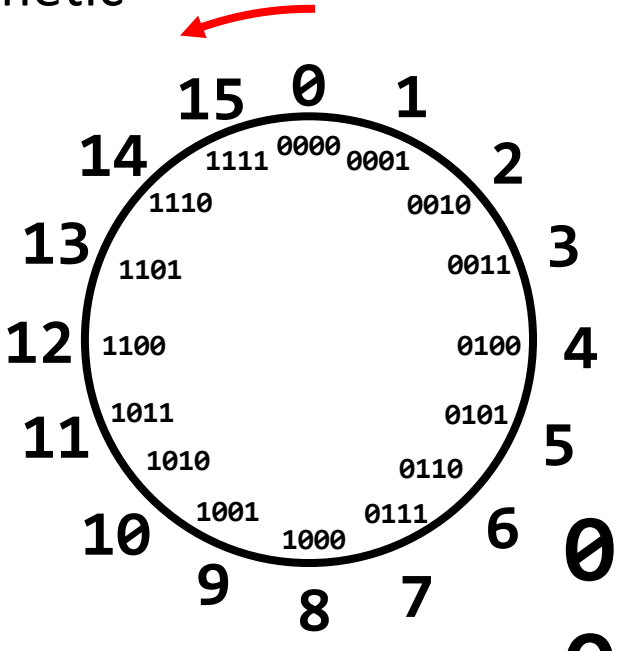
Finite numbers



- In computers, **numbers are finite.**
- Let's say our 4-digit display was counting up: 9997, 9998, 9999…
- What comes "next"?
  - What does this "0000" *really mean?*
  - It **wrapped around.**
- This is **overflow:** the number you are *trying* to represent is **too big to be represented.**
- Essentially, **all arithmetic on the computer is modular arithmetic!**
  - This causes a *lot* of software bugs.
  - https://en.wikipedia.org/wiki/Pac-Man#Level_256

Number carrousel

- Computers perform modulus arithmetic
  - Meaning: it goes around!
  - E.g. in a 4-bit computer

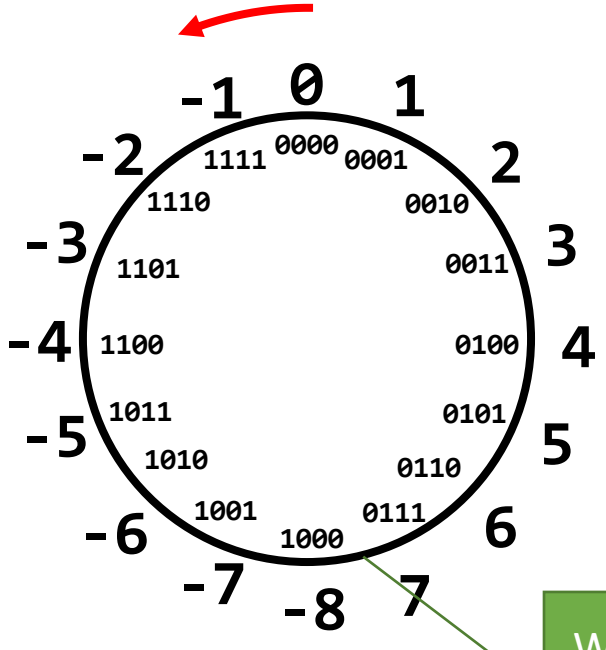



```
  0000
- 0001
------
  1111
```

what is 0 - 1?

What about negative carrousels?

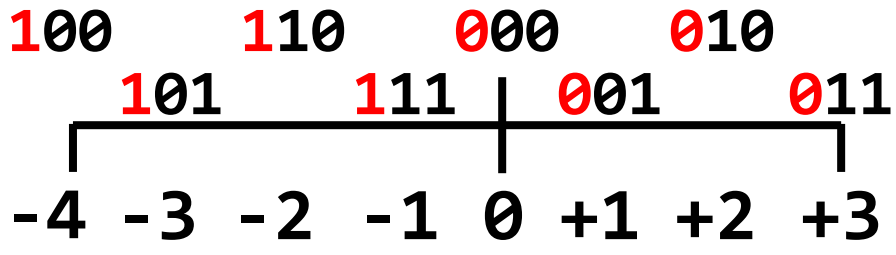
- How much is 0 – 1?
  - -1



Why did we stop here?

Signed Numbers (2's Complement)

- Representing negative numbers.
  - But it's a little strange!
- Hmm, it's a little lopsided: -4 **doesn't have a valid positive number**.

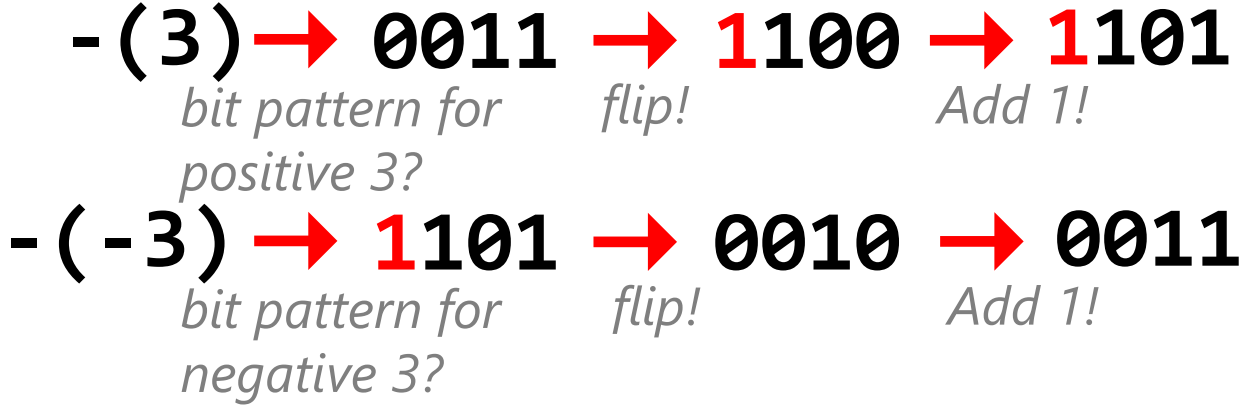


2's Complement

- I can tell it's negative if it starts with a 1 ☺
  - And if it's positive, then I can clearly see how much it's worth!
- But how exactly… can we tell the value of a negative number?
  - We need to negate (flip the sign) negative numbers
    - But HOW????

Two's complement arithmetic

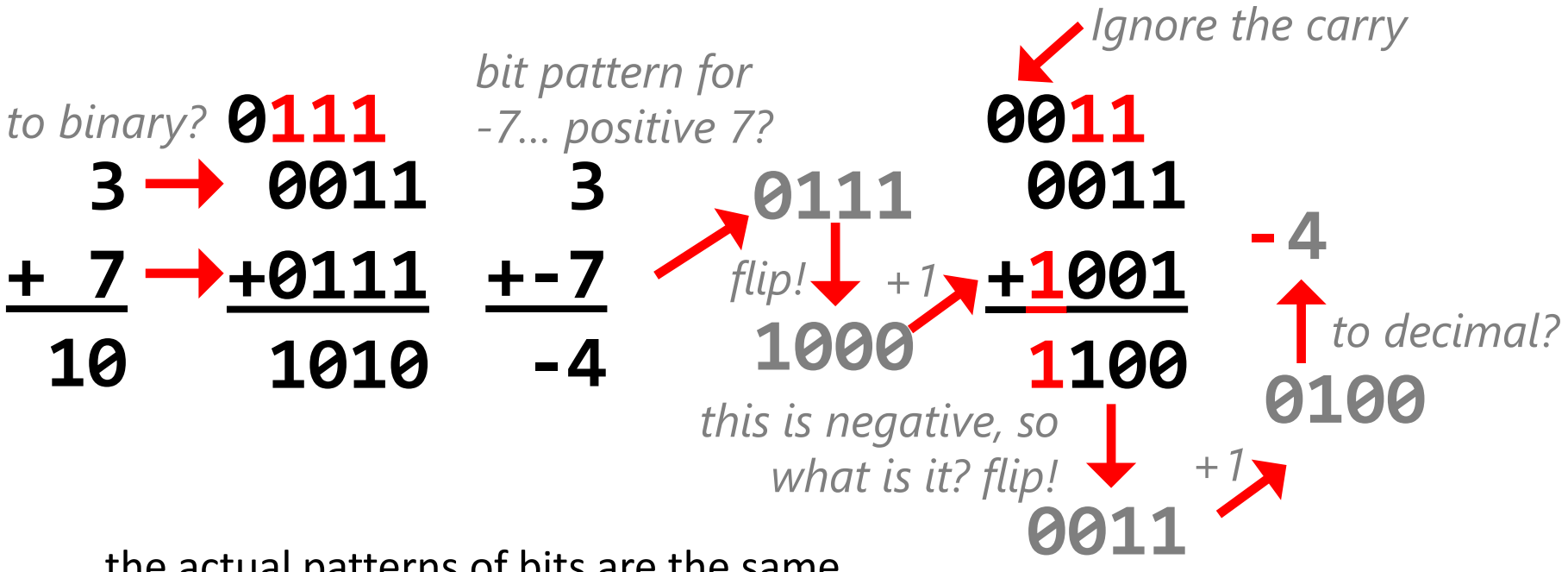
- Negation



- You don't need to subtract!!
  - flip(k)+1 == flip(k-1)
    - If you ignore the carry! ☺

Two's complement addition

- the great thing is: you can add numbers of either sign **without having to do anything special!**



the actual patterns of bits are the same.
so how does the computer "know" whether it's doing signed or unsigned addition?

Signed Numbers (2's Complement)

- Let's look some examples:

11010100 → -(00101011+1) → -(43+1) = -44

00100110 = 00100110 = 38

00000000 = 00000000 = 0

11111111 = -00000000 = -(0+1) = -1

- **If the MSB is 1**: Flip! Add one!
- **Otherwise**: Do nothing! It's the same!

Signed Numbers (2's Complement)

- What happens when we add zeros to a positive number:

```
0        10100110 =  ?
      -(01011001+1) =  ?
         -01011010 = -90
```

- What happens when we add ones to a negative number:

```
                  10100110 = -90
 11111111111111110100110 =
-0000000000001011001 = -90
```

Dang that's cool!

Can I Get an Extension?

- Sometimes you need to *widen* a number with fewer bits to more
- ***zero extension*** is easy: **put 0s at the beginning.**

$$1001_2 \rightarrow \textit{to 8 bits} \rightarrow 0000\ 1001_2$$

- But there are also **signed numbers… what about those?**
  - The **top bit (MSB)** of signed numbers determines **the sign (+/-)**
- ***sign extension*** puts ***copies of the sign bit*** at the beginning

$$1001_2 \rightarrow \textit{to 8 bits} \rightarrow 1111\ 1001_2$$

$$0010_2 \rightarrow \textit{to 8 bits} \rightarrow 0000\ 0010_2$$

Q: What happens when you sign extend the largest unsigned value?

Absolutely Bonkers

```
public class AbsTest {
  public static int abs(int x) {
    if (x < 0) {
      x = -x;
    }

    return x;
  }

  public static void main(String[] args) {
    System.out.println(
      String.format("|%d| = %d", Integer.MIN_VALUE, AbsTest.abs(Integer.MIN_VALUE))
    );
  }
}
                                                    // Outputs: |-2147483648| = -2147483648
```



Q: How many bits is a Java `int`? What happened here?

Integer Ranges

- Recall:
  - The range of an unsigned integer is 0 to $2^n - 1$
  - **Q**: Why do we subtract 1?

- What is the range of a 2's complement number?
  - Consider the sign bit, how many negative integers?
  - Consider, now, the positive integers.
  - Remember 0.

$$-2^{n-1} \text{ to } 2^{n-1} - 1$$

Q: What if you needed a range with far more negatives than positives?

Integers in ~~Java~~ C

- C allows for variables to be declared as either signed or unsigned.
  - Remember: "signed" does not mean "negative" just that it *can* be negative.

- An unsigned integer variable has a range from 0 to $2^n - 1$
- And signed integers are usually 2's complement: 2^{n-1} to $2^{n-1} - 1$
  - Where "n" is determined by the variable's size in bits.

- Integer Types: (signed by default, their sizes are arbitrary!!)

• `char`	`unsigned char`	`8 bits (byte)`
• `short int`	`unsigned short int`	`16 bits (half-word)`
• `int`	`unsigned int`	`32 bits (word)`
• `long int`	`unsigned long int`	`64 bits (double-word)`

- Usually no strong reason to use anything other than (un)signed int.

Q: What is the range of a `signed char`?

Integers in C: Limits

- Since sizes of integers are technically arbitrary…
  - They are usually based on the underlying architecture.

- … C provides standard library constants defining the ranges.
  - https://pubs.opengroup.org/onlinepubs/009695399/basedefs/limits.h.html

```c
#include <limits.h>              // Provides INT_MAX etc
#include <stdio.h>               // Provides printf

int main() {
  printf("%d ",  INT_MAX);       // Print the maximum signed int
  printf("%u\n", UINT_MAX);      // Print the maximum unsigned int
  return 0;
}                 // Output: 2147483647 4294967295
```

Q: Experiment with using "%d" for both. What is the result?

Casting

- C lets you move a value from an unsigned integer variable to a signed integer variable. (and vice versa)

- However, this is not always valid! Yet, it will do it anyway.
  - The binary value is the same, ***its interpretation is not***!
    - This is called ***coercion***, and this is a relatively simple case of it.
  - Since it ignores obvious invalid operations this is sometimes referred to as "weak" typing.
  - The strong/weak terminology has had very fragile definitions over the years and are arguably useless in our context. Let's ignore them.

- Moving values between different types is called ***casting***
  - Which sounds magical and it sometimes is.

Q: What is true of the result of casting the value -1 to an unsigned type?

Fractional Binary

Fractional numbers

- Up to this point we have been working with integer numbers.
  - Unsigned and signed!

2019

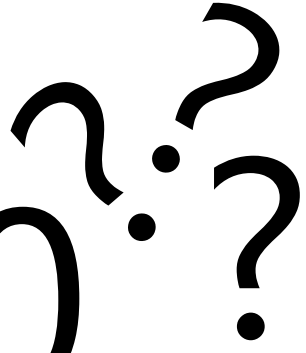
- However, Real world numbers are… Real numbers. Like so:

2019.320

- That create new challenges!
  - Let’s start by taking a look at them.

Just a fraction of a number

- The numbers we use are written **positionally**: the position of a digit within the number has a meaning.
- What about when the numbers go over the **decimal point**?



2	0	1	9	.	3	2	0
1000s	100s	10s	1s		10ths	100ths	1000ths
10^3	10^2	10^1	10^0		10^{-1}	10^{-2}	10^{-3}

A fraction of a bit?

- Binary is the same!
- Just replace 10s with 2s.

0 1 1 0 . 1 1 0 1

2^3	2^2	2^1	2^0		2^{-1}	2^{-2}	2^{-3}	2^{-4}
8s	4s	2s	1s		2ths?	4ths	8ths	16ths

To convert into decimal, just add stuff

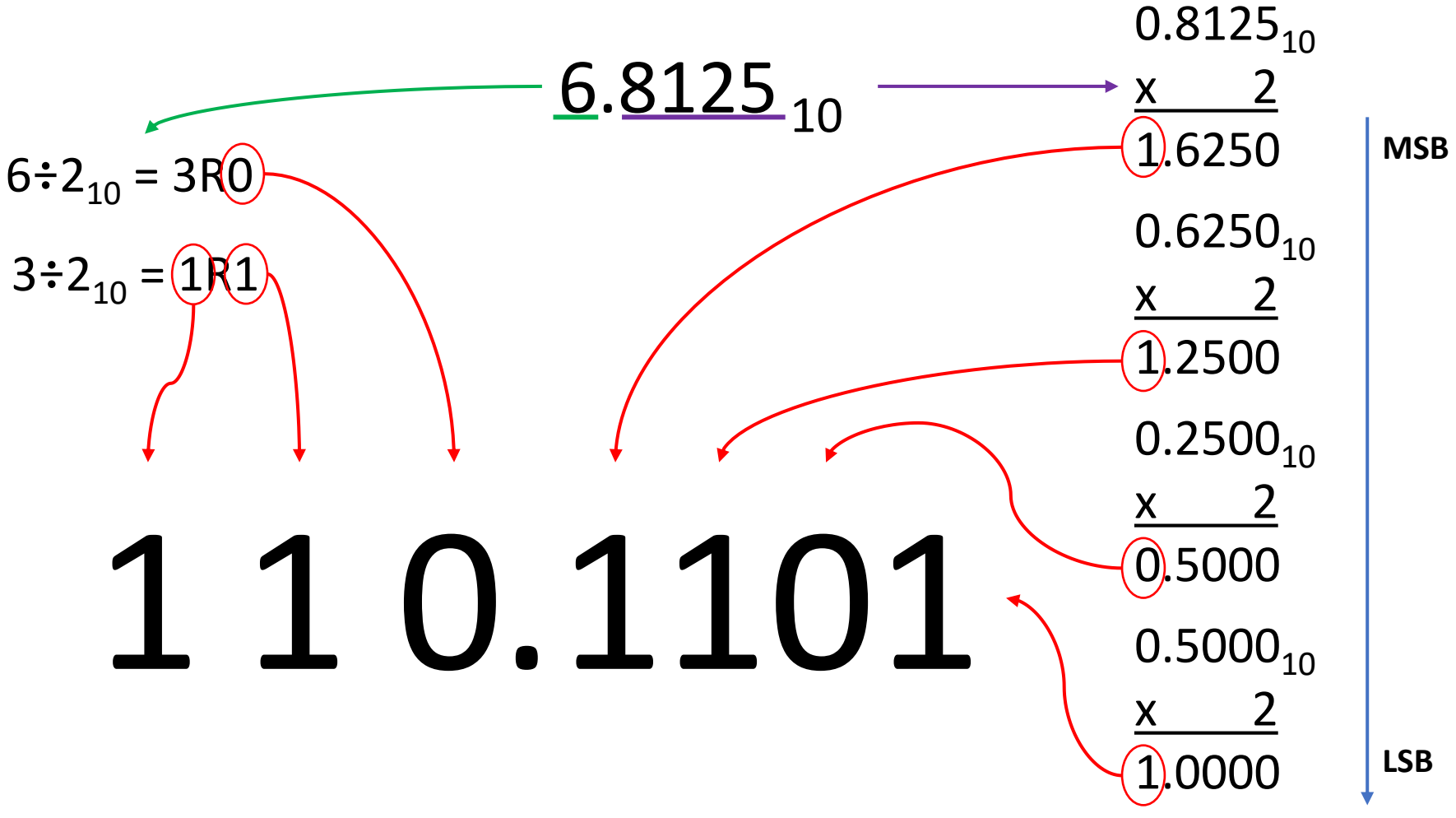
$$0\ 1\ 1\ 0\ .\ 1\ 1\ 0\ 1 =$$

2^3 2^2 2^1 2^0 2^{-1} 2^{-2} 2^{-3} 2^{-4}

$0 \times 8 +$
$1 \times 4 +$
$1 \times 2 +$
$0 \times 1 +$
$1 \times .5 +$
$1 \times .25 +$
$0 \times .125 +$
$1 \times .0625$

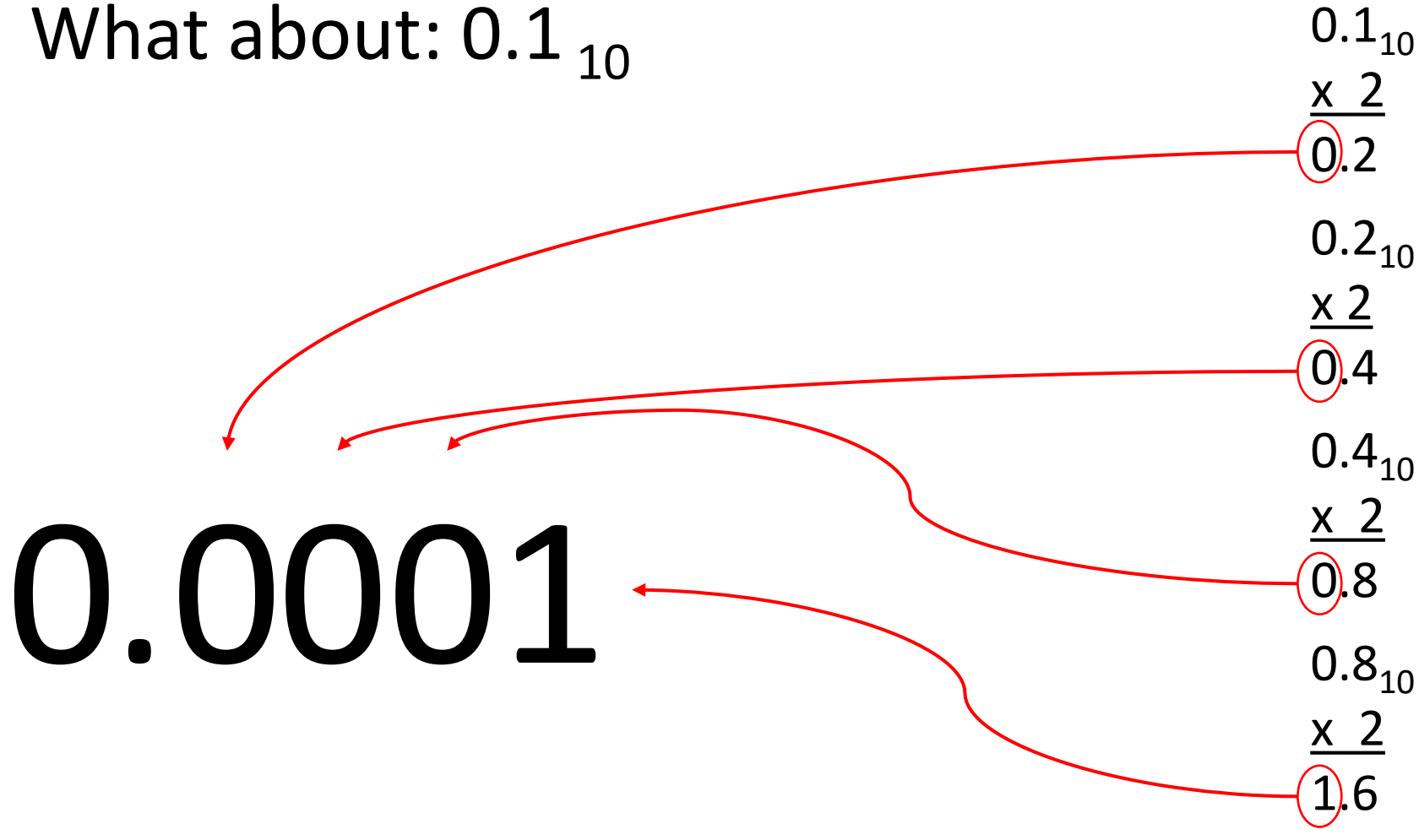
$= 6.8125_{10}$

From decimal to binary? Tricky?



So, it's easy right? Well…

What about: 0.1_{10}

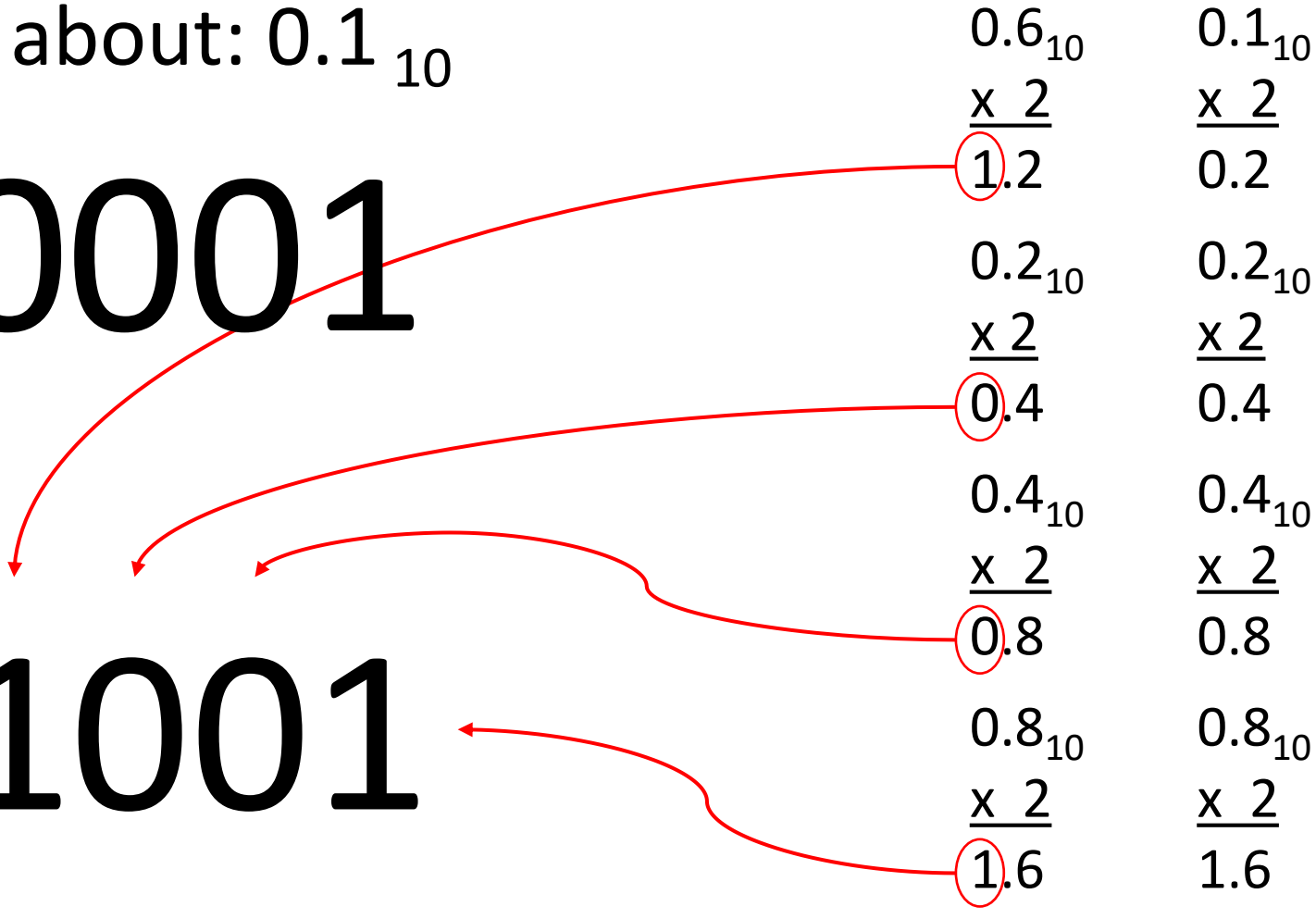


So, it's easy right? Well……

What about: 0.1_{10}

0.0001

1001



So, it's easy right? Well……..

What about: 0.1_{10}

0.0001
1001
1001
. . .

0.6_{10}	0.6_{10}	0.1_{10}
x 2	x 2	x 2
1.2	1.2	0.2
0.2_{10}	0.2_{10}	0.2_{10}
x 2	x 2	x 2
0.4	0.4	0.4
0.4_{10}	0.4_{10}	0.4_{10}
x 2	x 2	x 2
0.8	0.8	0.8
0.8_{10}	0.8_{10}	0.8_{10}
x 2	x 2	x 2
1.6	1.6	1.6

How much is it worth?

- Well, it depends on where you stop!

$0.0001_2 = 0.0625$

$0.00011001_2 = 0.0976\ldots$

$0.000110011001_2 = 0.0998\ldots$

Limited space!

- How much should we store?
  - We have 32-bit registers, so 32-bits?
    - Let's say we do!

- How many bits are used to store the integer part?
- How many bits are used to store the fractional part?



- What are the tradeoffs?